

0.1 Pixel and Normalized Coordinates

In standard computer vision, a pixel is represented by its coordinates (u, v) . However, working directly in pixels is numerically unstable for optimization. We transform these into *normalized coordinates* (u_n, v_n) relative to the optical center (C_X, C_Y) :

$$u_n = \frac{u - C_X}{C_X}, \quad v_n = \frac{v - C_Y}{C_Y} \quad (1)$$

This mapping ensures that the coordinates are centered at 0 and generally fall within the range $[-1, 1]$. Normalization is crucial because it decouples the image resolution from the mathematical optimization space, preventing gradient explosion.

0.2 The Ray Equation

A camera maps a 3D point to a 2D pixel. Inversely, a pixel (u, v) defines a 3D ray shooting out from the camera's optical center. In our model, the ray vector $\mathbf{r}$ is defined as:

$$\mathbf{r} = \begin{bmatrix} u_n \gamma_x \\ v_n \gamma_y \\ 1 \end{bmatrix} \quad (2)$$

Here, γ_x and γ_y act as the effective inverse focal lengths (scaled by C_X, C_Y). In a perfect pinhole camera, $\gamma_x = C_X / f_x$. By making γ variable across the image, we inherently model lens distortion without complex polynomials.

1 Bilinear Interpolation Mathematics

To model non-linear lens distortion smoothly, the image is divided into a grid (e.g., 3×3). Trainable parameters g_{ij} are placed at the nodes of this grid. For any arbitrary pixel (u, v) , the local gamma value is computed using bilinear interpolation.

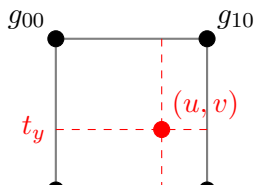
First, we map (u, v) to continuous grid coordinates (f_x, f_y) :

$$f_x = \frac{u}{\text{CELL_W}}, \quad f_y = \frac{v}{\text{CELL_H}} \quad (3)$$

We find the top-left integer node (i_x, i_y) and the fractional distances $t_x = f_x - i_x$ and $t_y = f_y - i_y$. The interpolated value $g(u, v)$ is a weighted sum of the four surrounding nodes:

$$g(u, v) = \underbrace{(1 - t_x)(1 - t_y)}_{w_{00}} g_{00} + \underbrace{t_x(1 - t_y)}_{w_{10}} g_{10} + \underbrace{(1 - t_x)t_y}_{w_{01}} g_{01} + \underbrace{t_x t_y}_{w_{11}} g_{11} \quad (4)$$

This approach guarantees C^0 continuity across the image, meaning there are no harsh jumps in distortion correction, while remaining computationally extremely cheap (Big-O $\mathcal{O}(1)$ per pixel).



3. **Decoupled Gradients:** The x -axis parameters strictly handle horizontal disparity (depth), while the y -axis parameters strictly handle epipolar alignment (camera rotation/vertical shift).

Thus, at each grid node, the parameter tensor Γ has shape $(R, C, 2)$, storing both γ_x and γ_y .

3 Stereo Geometry and Triangulation

Given a matched point in the left (u_L, v_L) and right (u_R, v_R) images, we generate two 3D rays: $\mathbf{v}_L$ and $\mathbf{v}_R$. Let Z be the depth along the left camera's optical axis, and Z_R be the depth along the right camera's axis.

The 3D point in the left camera frame is $P_L = Z\mathbf{v}_L$. The same point in the right camera frame is $P_R = Z_R\mathbf{v}_R$. The cameras are separated by a baseline vector $\mathbf{d} = [d_x, d_y, 0]^T$. The geometric relationship is:

$$P_L = \mathbf{d} + P_R \implies Z\mathbf{v}_L - Z_R\mathbf{v}_R = \mathbf{d} \quad (5)$$

In matrix form:

$$\underbrace{\begin{bmatrix} | & | \\ \mathbf{v}_L & -\mathbf{v}_R \\ | & | \end{bmatrix}}_{\mathbf{A}} \underbrace{\begin{bmatrix} Z \\ Z_R \end{bmatrix}}_{\mathbf{x}} = \mathbf{d} \quad (6)$$

Because rays in the real world rarely intersect perfectly due to noise, this system is overdetermined (3 equations, 2 unknowns). We solve it using the Ordinary Least Squares (OLS) method:

$$\mathbf{A}^T \mathbf{A} \mathbf{x} = \mathbf{A}^T \mathbf{d} \implies \mathbf{x} = (\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}^T \mathbf{d} \quad (7)$$

The estimated depth is simply $Z = x_1$.

4 Disparity Mathematics

Disparity is the difference in image coordinates of the same 3D point. In our ray space, the horizontal disparity is:

$$\text{disp} = v_{Lx} - v_{Rx} \quad (8)$$

From the triangulation equation, if $Z \approx Z_R$ (which is true when $Z \gg d_x$), we have $Z(v_{Lx} - v_{Rx}) \approx d_x$. Thus:

$$\text{disp} \approx \frac{d_x}{Z} \quad (9)$$

This fundamental inverse relationship explains why stereo vision is highly accurate for near objects (small $Z \implies$ large disparity) but loses precision for far objects (large $Z \implies$ disparity approaches zero).

5 Loss Function Deep Derivation

To train our Γ grid and baseline d , we define a custom loss function $\mathcal{L} = \mathcal{L}_{1D} + \mathcal{L}_Y$.

5.1 Why avoid Z -space loss?

A naive approach would be $\mathcal{L} = (Z_{pred} - Z_{true})^2$. However, since $Z \propto 1/\text{disp}$, a small pixel error at a large distance causes a massive Z error (Z^2 blow-up). This makes gradients highly unstable.

5.2 Relative Disparity Loss ($\mathcal{L}_{1D}$)

Instead, we optimize in ray-disparity space. The true expected ray disparity is $\text{disp}_{true} = d_x / Z_{true}$. We define the relative error as:

$$e_x = \frac{v_{Lx} - v_{Rx}}{\text{disp}_{true}} - 1 = \frac{(v_{Lx} - v_{Rx})Z_{true}}{d_x} - 1 \quad (10)$$

The loss is:

$$\mathcal{L}_{1D} = \frac{1}{2}e_x^2 \quad (11)$$

This relative formulation treats a 5% depth error at 1 meter the same as a 5% depth error at 10 meters, stabilizing the optimization.

5.3 Epipolar Y-Alignment Loss ($\mathcal{L}_Y$)

For perfect stereo, the vertical ray components should perfectly match the physical vertical baseline d_y .

$$e_y = (v_{Ly} - v_{Ry}) - \frac{d_y}{Z_{true}} \quad (12)$$

$$\mathcal{L}_Y = \frac{1}{2}e_y^2 \quad (13)$$

This forces the network to learn γ_y such that epipolar lines become perfectly horizontal in the ray space.

6 Huber Loss Robustness

Human clicks during calibration contain outliers. Standard Mean Squared Error (MSE) heavily penalizes large errors, pulling the model toward bad data points. We wrap our errors $e \in \{e_x, e_y\}$ in a Huber loss function $\mathcal{L}_\delta(e)$:

$$\mathcal{L}_\delta(e) = \begin{cases} \frac{1}{2}e^2 & \text{if } |e| \leq \delta \\ \delta \left(|e| - \frac{1}{2}\delta \right) & \text{if } |e| > \delta \end{cases} \quad (14)$$

With derivative:

$$\frac{\partial \mathcal{L}_\delta}{\partial e} = \begin{cases} e & \text{if } |e| \leq \delta \\ \delta \cdot \text{sign}(e) & \text{if } |e| > \delta \end{cases} \quad (15)$$

In the code, $\delta = 0.05$ (a 5% relative error threshold). Beyond this, the gradient magnitude is capped at δ , making the model highly robust to misclicks.

7 Gradient Derivations

Because we do not use automatic differentiation (like PyTorch), we must derive all gradients analytically using the chain rule.

7.1 Gradients with respect to Ray Vectors

Let $D = v_{Lx} - v_{Rx}$. Recall $e_x = \frac{DZ_{true}}{d_x} - 1$. Assuming $|e_x| \leq \delta$:

$$\frac{\partial \mathcal{L}}{\partial D} = e_x \frac{Z_{true}}{d_x} \quad (16)$$

Since $D = v_{Lx} - v_{Rx}$, we have:

$$\frac{\partial \mathcal{L}}{\partial v_{Lx}} = \frac{\partial \mathcal{L}}{\partial D}, \quad \frac{\partial \mathcal{L}}{\partial v_{Rx}} = -\frac{\partial \mathcal{L}}{\partial D} \quad (17)$$

Similarly for Y:

$$\frac{\partial \mathcal{L}}{\partial v_{Ly}} = e_y, \quad \frac{\partial \mathcal{L}}{\partial v_{Ry}} = -e_y \quad (18)$$

7.2 Gradients with respect to Grid Nodes (Γ)

By the ray equation $v_{Lx} = u_{nL} \gamma_{xL}$. Thus:

$$\frac{\partial v_{Lx}}{\partial \gamma_{xL}} = u_{nL} \quad (19)$$

Using bilinear interpolation, $\gamma_{xL} = \sum_{k=1}^4 w_k g_k$. Therefore, the gradient flows to the four surrounding grid nodes g_k :

$$\frac{\partial \mathcal{L}}{\partial g_k} = \frac{\partial \mathcal{L}}{\partial v_{Lx}} \frac{\partial v_{Lx}}{\partial \gamma_{xL}} \frac{\partial \gamma_{xL}}{\partial g_k} = \left(\frac{\partial \mathcal{L}}{\partial v_{Lx}} \right) \cdot u_{nL} \cdot w_k \quad (20)$$

This exact logic is implemented in the `forward_grads` Python function.

7.3 Gradients with respect to Baseline

$$\frac{\partial \mathcal{L}}{\partial d_x} = e_x \frac{\partial e_x}{\partial d_x} = e_x \left(-\frac{DZ_{true}}{d_x^2} \right) \quad (21)$$

$$\frac{\partial \mathcal{L}}{\partial d_y} = e_y \left(-\frac{1}{Z_{true}} \right) \quad (22)$$

8 Adam Optimizer Mathematics

We update the parameters using the Adam (Adaptive Moment Estimation) optimizer. For any parameter θ (which could be Γ or d) and its gradient g_t at epoch t :

1. Update biased first moment estimate (mean):

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (23)$$

2. Update biased second raw moment estimate (variance):

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (24)$$

3. Compute bias-corrected estimates:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \quad (25)$$

4. Update parameter:

$$\theta_t = \theta_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} \quad (26)$$

Where η is the learning rate, $\beta_1 = 0.9$, $\beta_2 = 0.999$, and $\epsilon = 10^{-8}$. Adam is ideal here because the gradients for Γ (which affect pixels) and d (which is in meters) have vastly different scales. Adam normalizes these scales automatically.

9 Regularization

Because the grid has 36 parameters (18 per camera) but we might only have 15 calibration points, the system is prone to overfitting. We add two regularization terms directly to the gradients.

9.1 Anchor Prior

We pull the Γ grid toward the theoretical initial value $G_0 = C_X / F_{init}$.

$$\nabla_{anchor} = \lambda_a (\Gamma - G_0) \quad (27)$$

This ensures that areas of the grid with no calibration points do not drift into absurd physical values.

9.2 Smoothness Laplacian

We enforce that adjacent nodes in the grid should have similar values. We compute the discrete 2D Laplacian using a 4-neighbor stencil:

$$\nabla^2 G_{i,j} \approx 4G_{i,j} - G_{i-1,j} - G_{i+1,j} - G_{i,j-1} - G_{i,j+1} \quad (28)$$

Adding $\lambda_s \nabla^2 G$ to the gradient acts as a physical tension, making the grid behave like an elastic sheet.

10 Fundamental Matrix and Epipolar Geometry

To assist the user in finding matching points, the software draws an epipolar line in the right image when the user clicks the left image.

The Fundamental Matrix F maps a point x_L to a line l_R via $l_R = Fx_L$. In our model, the local inverse intrinsic matrix at the clicked point is:

$$K_L^{-1} = \begin{bmatrix} \gamma_{xL}/C_X & 0 & -\gamma_{xL} \\ 0 & \gamma_{yL}/C_Y & -\gamma_{yL} \\ 0 & 0 & 1 \end{bmatrix} \quad (29)$$

Because the right pixel is unknown, we approximate K_R^{-1} using the mean Γ_R . The essential matrix E encodes the translation (baseline $\mathbf{d}$) via the skew-symmetric matrix:

$$E = [\mathbf{d}]_{\times} = \begin{bmatrix} 0 & -d_z & d_y \\ d_z & 0 & -d_x \\ -d_y & d_x & 0 \end{bmatrix} \quad (30)$$

The local Fundamental matrix is then computed as:

$$F = K_R^{-T} E K_L^{-1} \quad (31)$$

The epipolar line equation in the right image is $au_R + bv_R + c = 0$, where $[a, b, c]^T = F[u_L, v_L, 1]^T$.

11 NCC Matching Mathematics

To automatically suggest a matching point on the epipolar line, the code uses 1D Zero-Mean Normalized Cross-Correlation (ZNCC). Given an image patch A from the left image and B from the right image:

$$\text{ZNCC}(A, B) = \frac{\sum (A_i - \bar{A})(B_i - \bar{B})}{\sqrt{\sum (A_i - \bar{A})^2} \sqrt{\sum (B_i - \bar{B})^2}} \quad (32)$$

Where $\bar{A}$ and $\bar{B}$ are the mean pixel intensities of the patches. $\text{ZNCC} \in [-1, 1]$. By subtracting the mean, the match is invariant to global brightness changes. By dividing by the variance, it is invariant to contrast changes. The search is strictly 1D along the epipolar line $v_R = -(au_R + c)/b$.

After finding the integer pixel with the highest ZNCC, a parabola is fit to the scores of the peak and its two neighbors (s_{-1}, s_0, s_{+1}) to find the sub-pixel peak:

$$\Delta = \frac{s_{-1} - s_{+1}}{2(s_{-1} - 2s_0 + s_{+1})} \quad (33)$$

The refined coordinate is $u_{refined} = u_{best} + \Delta$.

12 Polynomial Post-Correction

Even after optimization, slight systematic biases may remain due to unmodeled physical factors (e.g., lens thickness, non-linear sensor scaling).

We absorb this residual error by fitting a quadratic polynomial mapping the predicted depth Z_{pred} to the true depth Z_{true} on the calibration set:

$$Z_{true} = a + bZ_{pred} + cZ_{pred}^2 \quad (34)$$

This is solved instantly using standard linear least squares, constructing a Vandermonde matrix from Z_{pred} .

13 Computational Complexity

The beauty of this pure-NumPy approach is its extreme efficiency.

- **Interpolation:** $\mathcal{O}(1)$ per point. 4 multiplications.
- **Triangulation:** $\mathcal{O}(1)$. Inverting a 2×2 matrix.
- **Training Loop:** For N points and E epochs, the complexity is $\mathcal{O}(N \cdot E)$. Since $N \approx 20$ and $E = 20000$, the total operations are on the order of 10^6 , taking less than a second on a modern CPU.
- **ZNCC Matching:** $\mathcal{O}(S \cdot P^2)$ where S is the search range (300 px) and P is the patch size (15 px). $\approx 67,000$ ops per click.

14 Numerical Stability

Several safeguards are built into the mathematics to prevent crashes:

1. **Parallel Rays:** If $v_{Lx} - v_{Rx} \approx 0$, the rays are parallel (infinite depth). The code catches $|denom| < 10^{-9}$ and returns NaN.
2. **Near-Singular Matrix:** If $\mathbf{A}^T \mathbf{A}$ is non-invertible, the code falls back to a purely 1D disparity equation $Z = d_x / denom$.
3. **Clipping:** Γ values are strictly clipped to $[0.05, 5.0]$ to prevent negative or infinite focal lengths.
4. **Minimum Baseline:** d_x is clamped to ≥ 0.02 meters to prevent division by zero in the loss function.

15 Possible Improvements

While highly effective, this model could be extended in a research setting:

- **Bundle Adjustment:** Jointly optimizing 3D point positions alongside the camera parameters to minimize reprojection error.
- **Multi-Scale Grids:** Using a hierarchical grid (e.g., 3×3 refining into 9×9) to capture high-frequency local lens distortions.
- **Uncertainty Estimation:** Computing the covariance matrix of $\mathbf{x} = (\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}^T \mathbf{d}$ to provide confidence intervals for the depth Z .

16 Final Intuition Chapter

To summarize the entire mathematical pipeline:

1. **Pixel Click:** The user selects (u_L, v_L) .
2. **Ray Generation:** The system looks up the local γ grid, interpolates, and shoots a 3D ray $\mathbf{v}_L$.
3. **Epipolar Search:** A fundamental matrix is built. The system searches along the resulting line in the right image using ZNCC to find (u_R, v_R) .
4. **Triangulation:** The intersection of $\mathbf{v}_L$ and $\mathbf{v}_R$ is computed via Least Squares to find raw Z .
5. **Correction:** Z is passed through a quadratic polynomial to yield the final, highly accurate physical distance.

A Full Mapping Between Python Functions and Math

Below is the direct translation between the mathematical concepts derived in this document and the exact Python functions in the provided source code.

- `bilinear(u, v, Gamma)` $\longleftrightarrow$ Implements Equation (4). Computes t_x, t_y and the four weights w_k .
- `build_ray(u, v, Gamma)` $\longleftrightarrow$ Implements Equations (1) and (2). Computes u_n, v_n and returns $\mathbf{r} = [u_n \gamma_x, v_n \gamma_y, 1]$.
- `triangulate(...)` $\longleftrightarrow$ Implements Equation (7). Constructs matrix $\mathbf{A}$, uses `np.linalg.solve(AtA, Atd)` to find Z .
- `forward_grads(...)` $\longleftrightarrow$ Implements Equations (10) through (21). Computes relative disparity loss, Huber clipping, and applies the chain rule analytically to return gradients for Γ_L, Γ_R , and d .
- `train(...)` $\longleftrightarrow$ Implements Adam Optimizer (Eq 22-25) and Regularization (Eq 26-27). Updates parameters in a loop.
- `fundamental_matrix_local(...)` $\longleftrightarrow$ Implements Equation (30). Computes K_L^{-1} , E , and F .
- `epipolar_guided_match(...)` $\longleftrightarrow$ Implements ZNCC (Eq 31) and Parabolic refinement (Eq 32).
- `fit_postcorrection(...)` $\longleftrightarrow$ Implements Equation (33). Solves Vandermonde matrix for a, b, c .